Carson Smith

Professor Minwoo Lee

ITCS 3156

May 1, 2026

Movie Recommendations Through Machine Learning

Introduction

One of the things that make social media and streaming services so addicting is their ability to bring content to the user that seems perfectly catered to the user's interest; that's because it is. As stated by Rahul Katravath in *What are Recommender Systems?* Recommender systems are "tools that suggest items to users based on their behavior, preferences, or past interactions." For example, on Netflix when you watch a movie, you'll notice that other similar movies will pop up because of recommender systems allowing users to easily find movies that'll interest them. This is needed because it'd be impossible for users to not only look through thousands of possible video/movie options but also be able to determine what they would want to watch without knowing beforehand. MovieLens is a website that collects data to make personalized movie recommendations. Some of this data was collected by Grouplens Research, which contains around 100,000 ratings and 3,600 tag applications applied to 9,000 movies by 600 users (specifically the small version, which I'll be using). In order to recommend movies for users, I'll be using a technique called "collaborative filtering," which, as described in the Google for Developers *Machine Learning* course, "uses similarities between users and items simultaneously to provide recommendations," which allows recommender systems to find C from knowing the user's relation to A and B.

I plan to use collaborative filtering as well as three machine learning algorithms in Pearson Correlation Collaborative Filtering, K-Nearest Neighbors (KNN), and a neural network to create a system that can either, based on a target film, find another similar film or, based on a target user, find the ten movies they're most likely to enjoy. Pearson correlation collaborative filtering was chosen for its ability to identify movies a user may enjoy based on similarities in movie rating patterns. The algorithm is used to measure how similar movies are rated by users,

allowing it to recommend movies that are closely related to a target movie. K-Nearest Neighbors was chosen to measure the similarity of movies to create recommendations based on similar rating patterns. It uses cosine similarity to measure the cosine of the angle between two rating vectors to find movies that are rated similarly by users. Movies with smaller cosine distances are considered more similar and therefore closer to the target movie. A neural network was chosen to predict the movies a target user would most likely enjoy based on learned movie and user patterns. It uses embedding layers to convert users and movies into vectors, which are combined using a dot product layer to find a predicted rating for the pair. By predicting the rating of all movies for a target user, the system recommends the 10 movies the user is most likely to enjoy.

Data

The MovieLens small dataset created by GroupLens Research includes 100,836 ratings on 9,742 movies from 610 users with ratings between 0.5 and 5 stars made by users. The ratings allow us to know how much a user enjoyed a particular movie, making this dataset well-suited for a recommender system since using it allows us to compare ratings on particular movies and see correlations between different film viewer bases. The dataset included multiple ".csv" files, of which the two I mainly used were ratings.csv and movies.csv. Ratings.csv held the ratings given by users for various movie IDs, while movies.csv held the movie titles and genres for various movie IDs. To get a better dataset, I merged these two files using their shared movieId column to get a dataset that contained all the relevant information, putting the movie titles with their ratings. I then grouped the movies by stats by creating a new dataset made of the average rating and number of times the movie was rated. I then used the count of ratings that every movie had to remove any movie with 50 or fewer ratings, cutting the number of movies from 9,179 to

437 to improve the recommender's reliability by focusing on only movies that had many ratings and therefore more ability to differentiate users' ratings and compare them.

Methods

The first machine learning algorithm I used was Pearson correlation collaborative filtering, which I used to find movies with similar user rating histories. I first made a matrix to store the user IDs, titles, and ratings. This matrix shows for every movie what a certain user rated it (if there was no rating, it would show NaN). Then, by taking an index of all the movie titles with over 50 ratings, I was able to filter that matrix so only more popular movies remain, creating my "filtered_matrix" variable. Then I used the system to target a specific movie (Toy Story (1995)), and using the filtered matrix, I was able to pull out the columns that held every user ID and their score of the film, which would act as my "target_ratings" to check if a film was close to my target movie. Then using ".corrwith()" I'm able to compare the target ratings vs. the filtered movie matrix. This returns the correlation between scores of other movies and scores of the target movie (Toy Story (1995)), and then I removed any rows with NaN scores since they don't show any useful information. Then I cleaned up the final data, removing the targeted movie before showing the top 10 recommendations based on movies rated similarly to the target movie.

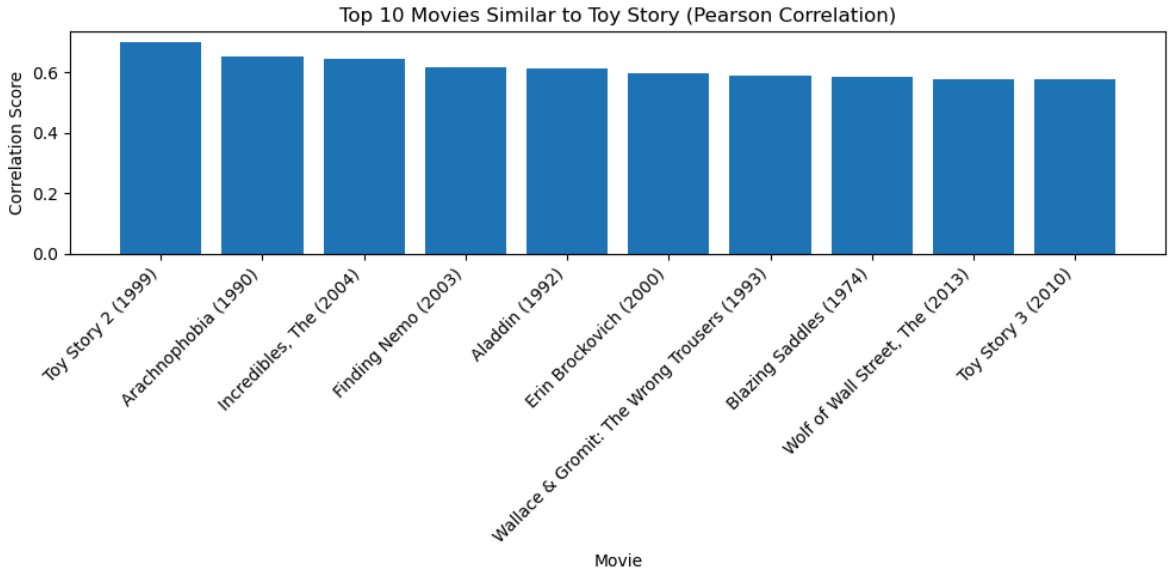
The second machine learning algorithm I used was K-Nearest Neighbors (KNN) to find the movies that are closest to the target movie (still Toy Story (1995)) using vectors in multidimensional space. I began by creating a matrix that held the rating of each movie for every user and filled any NaN spaces with 0 to allow distance calculating with KNN. Then I used the nearest neighbor model configuration to measure the similarity between movie rating vectors using cosine distance. Cosine distance measures the difference between vectors based on the

angle between them, with smaller angles showing higher similarity in rating patterns. Then I took the row of user ratings from the target movie (Toy Story (1995)) to compare it on the KNN model to find the top 10 movies (input as 11 neighbors because the movie itself was still on the list) and print them to find the top 10 smallest distances from the target movie to find which movies are the most similar.

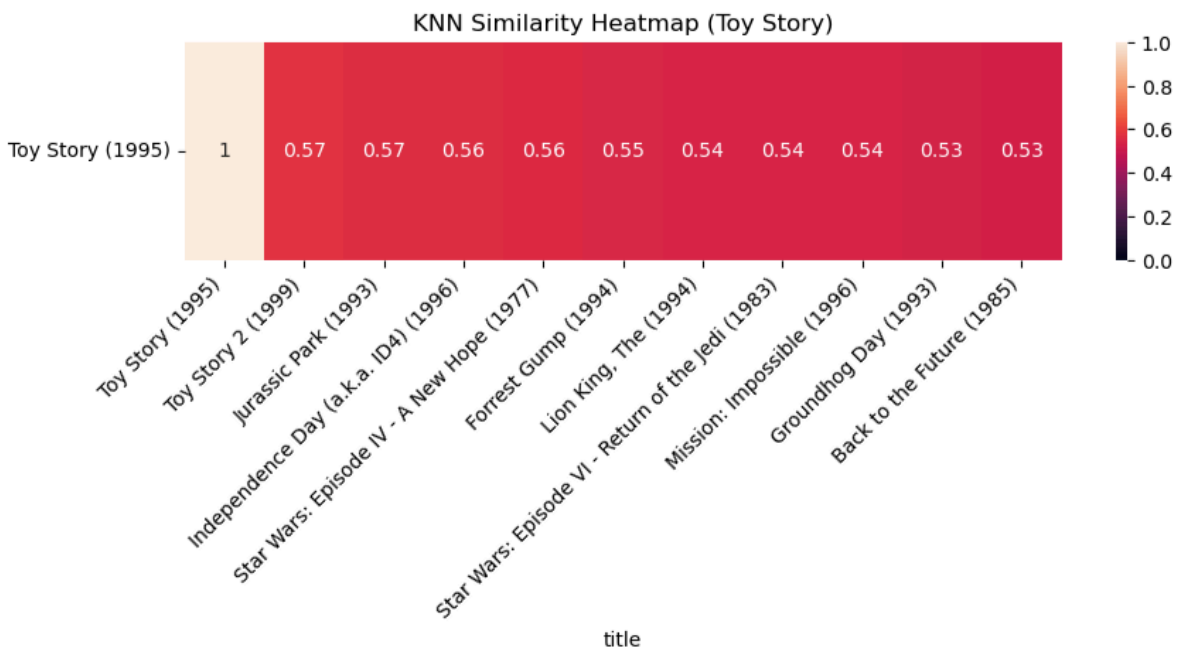
The third algorithm I used was a neural network that predicts how a user would rate a movie. First, user and movie IDs were converted into numbers, which were used to find the number of unique users and movies, which were used to set up the model. The model then turned each user and movie into a 50-dimensional vector using embedding layers. These vectors represent learned patterns about user preferences and movie features. Then a dot product was used to combine the two vectors and find a predicted rating. The model was built using user and movie inputs and the dot product as the output. It was trained using the ".fit()" function, which compares predicted ratings to actual ratings and updates the model over time to improve accuracy. After training, the model can predict ratings for any user and movie pair. A function was created to recommend movies by predicting ratings for all movies for a given user, then returning the top 10 highest-rated ones.

Results

I used 3 machine learning algorithms to analyze the ratings of movies made by MovieLens users. The first algorithm, Pearson Correlation, was given a target movie and looked for other movies that were similar to it based on user ratings. I used a top-10 correlation bar chart to see the similarities to the movie Toy Story. As seen in the chart below, many of the top movies are comedy movies similar to Toy Story, including its sequels; the biggest exception is Arachnophobia, which is actually a horror movie. Of the movies there are also Blazing Saddles and Wolf of Wall Street, which are much more adult-oriented movies rather than kids' movies like Toy Story is. The reason these seemingly random movies appear together is most likely because of how the system works because it is only going off ratings; a user who watches a varying amount of films could cause the algorithm to link different movies together through the same user.



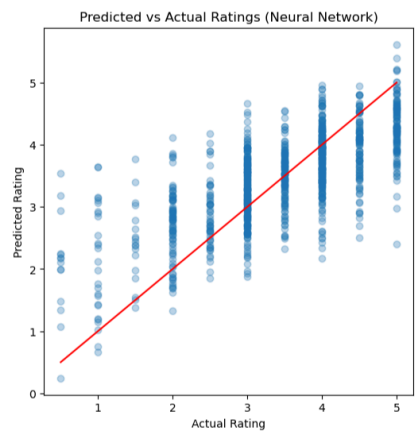
The second algorithm, KNN, was also given a target movie and was used to find the 10 movies that are most similar to it based on user ratings. Each movie was represented by a vector of user ratings, which could be used in cosine distance measurements to find similarity. When the distance is smaller, it'd indicate the movies were more similar. The algorithm would identify the top 10 movies with the smallest distance to the target movie. As seen below, the target movie was Toy Story, with the most similar movie being Toy Story with a 100% similarity, which is because they are the same movie. The rest of the films on here are a mix of more comedy and action films with no R-rated films, like in the Pearson correlation results. The lack of R-rated comedies happens because K-Nearest Neighbors is based on measuring distance between movie rating vectors rather than correlation; instead, it finds movies are similar if they are rated similarly by the same users.



The third algorithm used a neural network to predict what a target user would predict for every movie and then printed the top 10 of those recommendations. As seen below, this neural network's target was users 1 and 5. From these lists we can see that user 1 is recommended many more serious films than user 5, who is mostly recommended comedies. This is because the neural network personalizes recommendations by learning the user's preferences instead of determining them based on the similarity between movies.

```
Top recommendations for user 1:
Guess Who's Coming to Dinner (1967): 4.94
Man Bites Dog (C'est arrivé près de chez vous) (1992): 4.87
Incendies (2010): 4.82
Trial, The (Procès, Le) (1962): 4.73
Sophie's Choice (1982): 4.70
7th Voyage of Sinbad, The (1958): 4.69
Day of the Doctor, The (2013): 4.67
Beautiful Mind, A (2001): 4.67
Go (1999): 4.67
No Man's Land (2001): 4.66
```

```
Top recommendations for user 5:
Shawshank Redemption, The (1994): 4.71
Forrest Gump (1994): 4.61
Mr. Holland's Opus (1995): 4.61
White Squall (1996): 4.57
Much Ado About Nothing (1993): 4.56
Independence Day (a.k.a. ID4) (1996): 4.54
Clerks (1994): 4.52
Toy Story (1995): 4.51
Fugitive, The (1993): 4.47
Lion King, The (1994): 4.47
```



The chart to the right shows the predicted rating of movies vs. the actual rating in the neural network. Films above the red line are predicted to be higher rated than they actually are, while the opposite holds true for those below the red line. As we can see from movies rated 0.5 to 2.5, the neural network tends to overrate low-rated movies, while from 4.5 to 5, the movies seem to be underrated. This could be due to review websites; users typically only have a few perfect (5/5) or terrible (0.5-2.5) reviews to leave on movies, causing the neural network to not expect it. This is supported by 0.5 and 5 having fewer dots than the rest. The other oddity is that the neural network predicted ratings above 5 and below .5 for some films. This occurs because the neural network outputs continuous values from the dot product of embedding vectors and is not inherently constrained to the rating scale unless explicitly restricted. As a result, predictions can fall outside the valid rating range.

Conclusions

Through this project I explored how different machine learning algorithms could be used to build recommender systems using the MovieLens small dataset. I implemented and compared three recommendation approaches using Pearson correlation collaborative filtering, K-nearest neighbors (KNN), and a neural network recommender. The Pearson correlation model was effective at finding movies with similar overall rating behavior, although it also linked movies with very different genres due to overlapping user ratings. The KNN model produced more accurate recommendations by measuring distance between movie rating vectors, which resulted in recommendations that had more similar audience types and genres. The neural network provided the most personalized recommendations by learning patterns between users and movies through embeddings and predicted ratings. Unlike the other two models, it was able to generate unique recommendations for individual users rather than only finding movies similar to a target movie. Through this project, I learned how to make recommender systems, use collaborative filtering techniques, and how machine learning can be used to predict user preferences from large data sets. I also learned how different algorithms approached similarity in different ways, such as correlation, vector distance, and learning. I also gained experience preprocessing data, training neural networks, visualizing results, and interpreting machine learning outputs. Ultimately this project taught me how to create recommender systems similar to those used by streaming services and how to better interpret data from machine learning models.

Source Code

Github Link here

Works Cited

GroupLens. "Movielens." GroupLens, 29 July 2024, grouplens.org/datasets/movielens/.

"Collaborative Filtering | Machine Learning | Google for Developers." Google, Google, https://developers.google.com/machine-learning/recommendation/collaborative/basics Accessed 1 May 2026.

Katravath, Rahul. "What Are Recommender Systems?" GeeksforGeeks, GeeksforGeeks, 27 Dec. 2025, www.geeksforgeeks.org/machine-learning/what-are-recommender-systems/.

Mishra, Harsh. "User-Based Collaborative Filtering." GeeksforGeeks, GeeksforGeeks, 22 Jan. 2023, www.geeksforgeeks.org/machine-learning/user-based-collaborative-filtering/.

Sharma, Aakriti. "Recommender Systems Using KNN." GeeksforGeeks, GeeksforGeeks, 24 Oct. 2025, www.geeksforgeeks.org/machine-learning/recommender-systems-using-knn/.

Schifferer, Benedikt. "Using Neural Networks for Your Recommender System." NVIDIA Technical Blog, 28 Oct. 2024, developer.nvidia.com/blog/using-neural-networks-for-your-recommender-system/.